

Trabajo Integrador (Cuarto entregable)

DESARROLLO DE SOFTWARE SEGURO

Prof. Lic. Juan Pablo Villalba

Matias Leguizamon

Pablo Rodrigo Moya

Universidad de Gran Rosario

UGR



INDICE

Contexto del Proyecto..... 2

Nociones Preliminares.....4

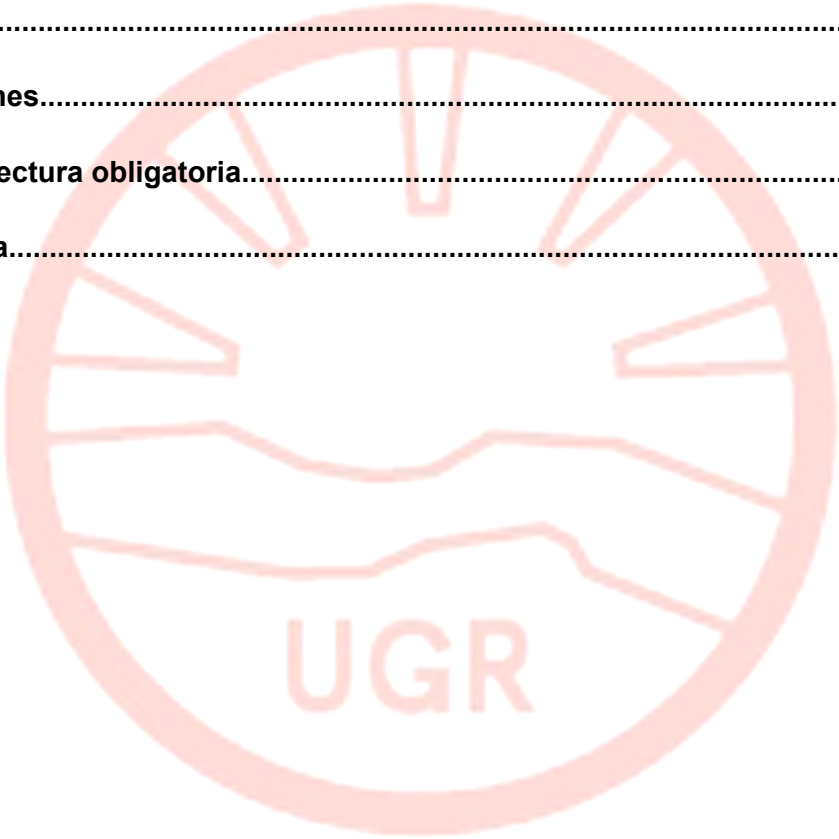
Pseudocódigo.....5

Código..... 5

Conclusiones.....6

Resumen lectura obligatoria..... 7

Bibliografía.....8





Ejercicio Interactivo de Desarrollo de Software Seguro: Creando una Aplicación para "FitLife"

Contexto del Proyecto

Imagina que eres parte del equipo de desarrollo de "FitLife", una empresa ficticia que ofrece una aplicación web para ayudar a las personas a llevar un estilo de vida saludable.

La aplicación permite a los usuarios registrarse, iniciar sesión, seguir sus rutinas de ejercicio y gestionar su dieta. Como parte del equipo, tu responsabilidad es asegurar que la aplicación sea segura y proteja la información sensible de los usuarios.

Objetivo del Ejercicio

El objetivo de este ejercicio es que los estudiantes aprendan a aplicar los principios de OWASP en el desarrollo de una aplicación web, utilizando pseudocódigo y código Python para implementar las mejores prácticas de seguridad.

Entregas Parciales

Entrega 1: Análisis de Riesgos y Diseño Seguro (Clase 1)

- Tarea: Investigar y discutir en grupos los 10 principales riesgos de OWASP. Cada grupo seleccionará uno o dos riesgos que podrían afectar a la aplicación "FitLife".
- Entregable: Un breve informe que describa los riesgos seleccionados y sus posibles impactos en la aplicación.
- Pseudocódigo y Python: Crear un pseudocódigo y código Python básico de las funciones de registro e inicio de sesión, incorporando medidas de seguridad basadas en OWASP.

Entrega 2: Validación y Sanitización (Clase 2)

- Tarea: Implementar funciones de validación y sanitización de entradas de usuario.



- Entregable: Código de las funciones de validación y un informe sobre la importancia de la validación de entradas.
- Pseudocódigo y Python: Crear pseudocódigo y código Python para la validación de entradas.

Entrega 3: Autenticación y Manejo de Sesiones (Clase 3)

- Tarea: Implementar un sistema de autenticación seguro y manejo de sesiones.
- Entregable: Código de las funciones de autenticación y un informe sobre las mejores prácticas de gestión de sesiones.
- Pseudocódigo y Python: Crear pseudocódigo y código Python para el manejo de sesiones.

Entrega 4: Pruebas de Seguridad (Clase 4)

- Tarea: Realizar pruebas de seguridad en la aplicación "FitLife" y documentar las vulnerabilidades encontradas.
- Entregable: Informe de pruebas con resultados y correcciones implementadas.
- Pseudocódigo y Python: Estrategias de pruebas de seguridad.

Entrega 5: Implementación de Medidas de Seguridad (Clase 5)

- Tarea: Implementar medidas de seguridad adicionales basadas en los aprendizajes de las clases anteriores.
- Entregable: Código de las medidas de seguridad implementadas y un informe sobre cómo estas medidas ayudan a mitigar los riesgos de seguridad.
- Pseudocódigo y Python: Crear pseudocódigo y código Python para las medidas de seguridad adicionales.

Entrega Final: Aplicación Funcional y Segura (Clase 6)

- Tarea: Entregar la aplicación "FitLife" completa, funcional y segura, implementando todas las medidas de seguridad aprendidas.



- Entregable: Código Python de la aplicación "FitLife" que incluya todas las funcionalidades y prácticas de seguridad basadas en OWASP.

Evaluación

Los estudiantes serán evaluados en función de:

- La calidad de sus entregables.
- La aplicación de prácticas de seguridad basadas en OWASP.
- La claridad y lógica del pseudocódigo y código Python.
- La funcionalidad y seguridad de la aplicación final.
- La colaboración y participación en las actividades grupales.

Conclusión

Este ejercicio interactivo no solo permite a los estudiantes aprender sobre OWASP y las vulnerabilidades comunes en aplicaciones web, sino que también les brinda la oportunidad de aplicar este conocimiento en un contexto práctico y colaborativo. Al trabajar en un proyecto realista como "FitLife" y utilizar tanto pseudocódigo como código Python, los estudiantes desarrollarán habilidades críticas para su futura carrera en el desarrollo de software seguro.

Requisitos:

Carátula, índice, cuerpo, conclusiones, referencias (APA)

Nociones preliminares

Teniendo en cuenta la cantidad abismal de posibilidades de vulnerar una pagina web, en especifico una tan sencilla donde las tecnicas empleadas para hacer de la misma una app segura rozaron lo sencillo, el enfoque de este entregable se da frente a tres (3) metodos de



los cuales 2 están orientados a evitar ataques DDoS y uno (1) a reforzar la seguridad frente a inyección sql.

Pseudocódigo:

```

1  FUNCIÓN detectar_inyección_sql (string_digitado):
2      patrones_más_usados = ["copiar y pegar los más usados"]
3      for patron in string_digitado:
4          if patron in string_digitado:
5              alerta "posible inyeccion"
6              return True
7      return false
8
9  @ruta registro AND @ruta login
10 if detectar_inyeccion_sql(password o username) = True
11     return redirect
12
13 -----
14
15 import flask_limiter
16 definimos valores por defecto a 200 por día, 50 por hora
17 agregamos @limiter("10 por minuto") en la ruta @register y @login antes de cada Def register/login
18
19 -----
20
21 en @ruta login
22 if usuario :
23     intentos fallidos = usuario(x)
24     tiempo de bloqueo = usuario(z)
25
26     if tiempo de bloqueo y hora actual < mayor que < tiempo de bloqueo
27         alerta = "cuenta bloqueada temporalmente"
28
29     corroborar que el login se haya dado correctamente
30 else:
31     intentos fallido +=1
32     if intento fallidos >= 5
33         aumentas tiempo de bloqueo + 15 minutos

```

Código:

```

15 limiter = Limiter( #4 establecemos los limites de tiempo por defecto a las pretciones
16     get_remote_address,
17     app=app,
18     default_limits=["200 per day", "50 per hour"]
19 )

```

```

75 def detectar_inyeccion_sql(input_string): #4 definimos la función para usarla más tarde como
76     patrones_inyeccion = ["'", '"', ";", "--", "/*", "*/", "xp_"] #4 patrones más usados
77     for patron in patrones_inyeccion: #4 bucle para buscsar cada opción de arriba
78         if patron in input_string:
79             flash('Posible ataque de inyección SQL detectado.')
80             return True
81     return False

```

```

75 def detectar_inyeccion_sql(input_string): #4 definimos la función para usarla más tarde como
76     patrones_inyeccion = ["'", '"', ";", "--", "/*", "*/", "xp_"] #4 patrones más usados
77     for patron in patrones_inyeccion: #4 bucle para buscsar cada opción de arriba
78         if patron in input_string:
79             flash('Posible ataque de inyección SQL detectado.')
80             return True
81     return False

```



```

110 @app.route('/login', methods=['POST'])
111 @limiter.limit("10 per minute") #4 volvemos a definir 10 peticiones x minuto
112 def login():
113     username = request.form['username']
114     password = request.form['password']
115
116     if detectar_inyeccion_sql(username) or detectar_inyeccion_sql(password):
117         return redirect(url_for('index')) #4 usamos la función nuevamente para evitar inyecciones
118
119
130 if user:
131     failed_attempts = user[2] #4 user [0] es user, 1 es password, 2 es intentos fallidos, y 3 es tiempo de
132     #bloqueo. por eso se usa [2] y [3] respectivamente en estas dos lineas
133     lockout_time = user[3]
134
135     if lockout_time and datetime.now() < lockout_time: #4 si el tiempo de bloqueo y la hora es mayor al
136     #tiempo de bloqueo, simplemente salta la alerta de que la cuenta sigue bloqueada
137         flash('Cuenta bloqueada temporalmente. Intenta nuevamente más tarde.')
138         return redirect(url_for('index'))
139
140
141 if user[1] == password:
142     user_obj = User(username=user[0], password=user[1])
143     session.clear()
144     login_user(user_obj)
145     session.permanent = True
146     database.execute('UPDATE usuarios SET failed_attempts = 0, lockout_time = NULL WHERE username = ?',
147     (username,))
148     database.execute('COMMIT')
149     flash('¡Has iniciado sesión exitosamente!')
150     return redirect(url_for('dashboard'))
151
152 else:
153     failed_attempts += 1 #4 si linea 138 no se cumple entonces hacemos +1 a user[2]
154     if failed_attempts >= 5: #4 si hay 5 intentos fallidos almacenados en user[2] => agregamos 15
155     minutos de bloqueo a user[3]
156         lockout_time = datetime.now() + timedelta(minutes=15)
157         database.execute('UPDATE usuarios SET failed_attempts = ?, lockout_time = ? WHERE username = ?
158         ', (failed_attempts, lockout_time, username))
159     else:
160         database.execute('UPDATE usuarios SET failed_attempts = ? WHERE username = ?',
161         (failed_attempts, username))
162         database.execute('COMMIT')
163         flash('Usuario o contraseña incorrectos. Intenta nuevamente.')

```

Conclusiones:

Si bien ya habíamos planteado un regex para evitar lo mayor posible el exponer la aplicación FitLife frente a ataques de inyección sql, notamos que no fue tan efectivo como pensábamos que lo había sido en su primera instancia, para complementar quisimos agregar apartados donde se agregaban límites de peticiones a la aplicación, además de un contador de cantidad de peticiones con tal de evitar en su medida la explotación por parte de un ataque de denegación de servicios.



Para correr en local el FitLife se necesita de instalar en la computadora: Python, Flask, sqlite4, flask_login, flask_wtf, re

Tras correr el programa por Visual Studio Code, y para correr en local FitLife se necesita ingresar a : <http://127.0.0.1:5500/FitLife-Test/template/index.html>

Resumen lectura obligatoria:

Secure Software System

Gestionar el ciclo de vida del desarrollo de sistemas de software seguros es una tarea compleja que requiere la integración de diversas áreas de gestión. Cuanto más comprendan los miembros del equipo cómo cada área influye en el éxito del producto, más efectivos serán en contribuir a entregar un producto que cumpla con los requisitos del cliente.

Existen diferentes metodologías de desarrollo que intentan integrar estas áreas de gestión en un enfoque integral aplicable a lo largo del ciclo de vida. Debido a la naturaleza de los sistemas de software, algunas metodologías son más efectivas para ciertos esfuerzos de desarrollo que otras. Sin embargo, a veces, estas metodologías o su aplicación pueden descuidar aspectos de las áreas de gestión, perjudicando el esfuerzo de desarrollo.

El seguimiento de requisitos, la gestión de cambios y la gestión de problemas son ejemplos de procesos y desafíos que deben ser gestionados a lo largo del ciclo de vida del desarrollo de sistemas de software seguros.

—

Software Security Concepts & Practices

Las amenazas al software son variadas y rápidas. La era digital actual abre la puerta a diversas amenazas de seguridad que ponen a las organizaciones en situaciones de explotación diariamente. Gestionar estas situaciones es un trabajo complejo y desafiante para los expertos en seguridad. Por lo que se entiende, se intenta resumir el estado actual de las estrategias de desarrollo de software y sus recursos frente a estas amenazas.



La situación actual indica que es necesario clasificar primero las diversas amenazas en el contexto del desarrollo seguro, luego analizar su impacto y, finalmente, mitigarlas para producir software y sistemas organizacionales efectivos y seguros. Se busca proporcionar una conciencia situacional breve y efectiva sobre las diversas amenazas y su impacto en el desarrollo seguro. Además, nos topamos con mucha información sobre estrategias de mitigación efectivas para abordar los problemas de seguridad en las organizaciones.

Bibliografía:

<https://flask-limiter.readthedocs.io/en/stable/>

"Secure Software System" Capítulo 5

"Software security concepts & practices" (Pág 50 a 67)